Findings by priority

Priority	Count
High	4
Medium	8
Low	4
Total findings	**16**
Elements with no distinct threat	5
Elements in coverage checklist	45

Rank	Finding ID	Affected component / data flow	ATT&CK category (tactic)
1	F01	DF1 EE1→P1 Administrator workstation to SSH bastion (continuing through DF2, DF14)	Initial Access
1	F02	DF10 P4→EE3 Tenant 1 cluster-wide credential (delivered via DF3 and the app1/app2 main branch in DS3)	Privilege Escalation
3	F03	DF7 → DS1 Redis rendered manifest cache	Impact
3	F04	DS2 Ops persisted Secrets (via DF9)	Credential Access
5	F05	DF7 → DS1 Redis (read or sniff)	Collection
5	F06	DF5 P3→P8 Git fetch over HTTP	Credential Access
7	F07	P2 Argo CD API Server (local admin account)	Credential Access
7	F08	DF11 and DF12 P4→EE4/EE5 app3 and app4 namespace credentials (via DF3)	Execution
9	F09	P8 Gitea server / DS3 (authorization at TB4)	Privilege Escalation
9	F10	P4 Application Controller / DF10–DF12 (auto-prune and self-heal disabled)	Persistence
11	F11	P1 SSH bastion (TB1)	Initial Access
11	F12	DS4 Local Argo, Gitea and Kubernetes logs (DF15)	Defense Evasion
13	F13	DF3 EE2→P8 developer Git push over HTTP	Credential Access
14	F14	DS3 repositories app1, app3, app4 (anonymous read, TB4)	Collection

14	F15	P8 Gitea server and DS1 Redis (via DF5 and DF7)	Impact
14	F16	DF13 P6→EE6 notification SMTP (TB11)	Exfiltration

Notes: Competition ranking (tied findings share a rank; the next rank skips). Source: Argo C

ATT&CK technique
Valid Accounts (T1078)
Valid Accounts (T1078), abusing an over-scoped credential; payload via Deploy Container (T1610)
Stored Data Manipulation (T1565.001)
Unsecured Credentials (T1552)
Data from Information Repositories (T1213); Network Sniffing (T1040)
Adversary-in-the-Middle (T1557)
Brute Force (T1110)
Deploy Container (T1610)
Exploitation for Privilege Escalation (T1068) or Valid Accounts (T1078)
Deploy Container (T1610), retained workloads
Exploit Public-Facing Application (T1190)
Indicator Removal (T1070)
Adversary-in-the-Middle (T1557)
Code Repositories (T1213.003)

Network Denial of Service (T1498)
Exfiltration Over Alternative Protocol (T1048)

CD GitOps Delivery system package R

Threat mechanism and consequence
A stolen administrator SSH key lets the attacker authenticate to the bastion, which has no extra MFA. The attacker then tunnels HTTPS to the Argo API Server and Ops Kubernetes API (DF2) and runs kubectl against all three tenant control planes (DF14).
A developer pushes a malicious manifest to main. With no approval or signing and auto-sync on, P4 applies it under the cluster-wide credential, so it can exceed one namespace (privileged workload or cluster-level RBAC), giving control of Tenant 1 including the other app's namespace (TB8).
Crafted entries written to the unauthenticated, plaintext Redis make P4 apply manifests that never passed through Git. Tenant 1's cluster-wide credential makes this most damaging, and no commit exists to show it.
DS2 holds per-tenant bearer-token Secrets, the app2 repo credential, notification credentials and the retained initial admin password, not presumed encrypted at rest. Anyone with Secret read gets direct Tenant 1 cluster-wide access that bypasses Argo RBAC, AppProjects and Argo audit, plus Argo admin login.
Reading or sniffing Redis exposes rendered manifests. They may contain Kubernetes Secret values or embedded credentials, and they reveal deployment topology.
An on-path attacker captures the app2 repository credential sent over HTTP and/or alters manifests as P3 fetches them, giving private-repo access and manifest injection, including toward Tenant 1.
Local admin stays enabled, SSO is off and the bootstrap password Secret remains. A user who can reach the login may guess or replay the admin password, gaining Argo admin and the ability to define and sync Applications across tenants.
A malicious or stolen developer account pushes an unreviewed manifest to main, and auto-sync deploys attacker workloads into the app3 or app4 namespace. It can run attacker code and reach in-namespace data; namespace-scoped RBAC and AppProject limits keep it inside that namespace.
With no signing or approval, manifest integrity rests entirely on Gitea authorization. If an attacker gets write beyond their own repository, they can alter app1/app2 manifests and reach Tenant 1 cluster-wide, or alter any application.
A malicious resource stays deployed after its commit is reverted, because pruning is off. Live changes made directly in a namespace are not reverted either. The implant survives until someone notices an out-of-sync status.
Code execution on the only Internet-reachable host would expose administrator authentication material and tunneled or kubectl sessions. It would also give a private-network position toward the Argo API Server, Ops API and tenant control planes.
With no central immutable archive, an actor with admin access can delete or alter local Argo, Gitea and Kubernetes logs and events. That removes evidence of malicious syncs and credential use.
An observer on the developer's path captures a Gitea credential and pushes as that developer, leading to F02 or F08 outcomes.
Any workload that reaches Gitea can clone three repositories without authentication. It sees manifests, history and topology, plus any embedded secrets, which helps plan attacks such as F02 and F08.

Flooding Gitea or Redis stalls fetch and manifest generation. Rollouts and changes stop, but running applications are unaffected.
An admin who can alter notification content or use the stored notification credentials can send data out through the trusted TLS email path.

ev 1.0 and diagram sheets only; no outside sources used.

Required initial attacker access	Additional assumption	Priority
Holder of a leaked admin SSH key, or compromised administrator workstation	None	**High**
Organization developer with write access to the app1 or app2 repository	The AppProject limits repositories and destinations only. It does not block cluster-scoped kinds or privileged pod specs.	**High**
Compromised tenant workload (ordinary network reachability)	A route from the foothold to Redis exists (no NetworkPolicy, shared security group). P4 trusts cached entries without re-rendering.	**High**
Malicious or compromised administrator with Ops kubectl (e.g., after F01), or a leaked Argo credential that has Secret-read	None on the admin path. A non-admin leaked identity must hold Secret-read RBAC.	**High**
Compromised tenant workload	Same Redis reachability as F03. Cached content actually includes sensitive values (the package says 'potentially').	**Medium**
Foothold that can observe or modify traffic between Ops pods and Gitea (e.g., a compromised tenant workload)	On-path position is achievable inside the VPC. The credential is sent in a replayable form.	**Medium**
Low-privilege Argo user with private access to P2	The password is weak or never rotated, and no effective login throttling exists.	**Medium**
Developer with write access to the app3 or app4 repository, or a leaked developer credential	None	**Medium**
Developer with write access to one repository	A Gitea permission bypass, misconfiguration or admin-account compromise. None is established.	**Medium**
Developer with repo write, or compromised tenant workload	For the workload path, its service account can create or modify objects in its namespace. Responders revert only Git.	**Medium**
Anonymous external attacker	An exploitable flaw in a bastion-exposed service. Key authentication alone is not brute-forceable under the package's controls.	**Medium**
Malicious or compromised administrator	Admin-level access can modify local log storage.	**Medium**
Foothold on the approved private path to Gitea	On-path position on connectivity the package does not model. The HTTP authentication is replayable.	**Low**
Compromised tenant workload	Route to Gitea from the foothold. Repositories contain sensitive details.	**Low**

Compromised tenant workload	Route to the targets and no effective rate or capacity limits.	**Low**
Malicious or compromised administrator	Notification templates or config can be edited to include data of interest.	**Low**

Priority justification
A single-factor gate to the highest privilege at every tier.
One actor reaches production automatically under cluster-wide privilege.
No authentication barrier, and the result is unreviewed deployment.
Yields a cluster-wide token and an admin password; the tokens keep working after the attacker leaves.
Confidentiality only, and it depends on cache content.
High impact, but an on-path position is not established.
Severe if it succeeds, but it needs unestablished weaknesses.
No approval gate, but blast radius is bounded to one namespace.
High impact, but it depends on an unestablished control failure.
Extends dwell time for other attacks.
Highest exposure, but it needs an unestablished flaw.
Causes no compromise itself, but undermines detection and attribution for every other finding.
Depends on the unmodeled network path.
Read-only, and anonymous in-VPC reads are by design.

Affects the delivery plane only.
One fixed destination, and an admin has easier paths.

Element ID	Type
EE1	External entity
EE2	External entity
EE3	External entity
EE4	External entity
EE5	External entity
EE6	External entity
P1	Process
P2	Process
P3	Process
P4	Process
P5	Process
P6	Process
P7	Process
P8	Process
P9	Process
DS1	Data store
DS2	Data store
DS3	Data store
DS4	Data store
DF1	Data flow
DF2	Data flow
DF3	Data flow
DF4	Data flow
DF5	Data flow
DF6	Data flow
DF7	Data flow
DF8	Data flow
DF9	Data flow
DF10	Data flow
DF11	Data flow
DF12	Data flow
DF13	Data flow
DF14	Data flow
DF15	Data flow
TB1	Trust boundary
TB2	Trust boundary
TB3	Trust boundary
TB4	Trust boundary
TB5	Trust boundary
TB6	Trust boundary
TB7	Trust boundary
TB8	Trust boundary
TB9	Trust boundary
TB10	Trust boundary

TB11	Trust boundary

The checklist is not part of the findir

Name / endpoints
Administrator workstation
Application team developer
Tenant 1 Kubernetes API app1 app2 cluster wide credential
Tenant 2 Kubernetes API app3 namespace credential
Tenant 3 Kubernetes API app4 namespace credential
Notification email service
SSH bastion
Argo CD API Server
Repository Server
Application Controller
ApplicationSet Controller
Notifications Controller
Dex installed SSO inactive
Gitea server
Ops Kubernetes API
Redis rendered manifest cache
Ops Kubernetes persisted Secrets ConfigMaps and custom resources
Gitea repositories and account data
Local Argo Gitea and Kubernetes logs
EE1 to P1
P1 to P2,P9
EE2 to P8
P8 to DS3
P3 to P8
P2,P4 to P3
P2,P3,P4 to DS1
P2,P4,P5,P6 to P9
P9 to DS2
P4 to EE3
P4 to EE4
P4 to EE5
P6 to EE6
P1 to EE3,EE4,EE5
P2,P3,P4,P5,P6,P8,P9 to DS4
Public Internet to bastion public subnet
Ops cluster perimeter and private subnet to bastion, teams, tenant clusters, notification service
argocd namespace to other Ops services
gitea namespace and repository authorization
Tenant 1 cluster
Tenant 2 cluster
Tenant 3 cluster
app1 and app2 namespaces within Tenant 1
app3 namespace in Tenant 2
app4 namespace in Tenant 3

Ops services to external notification service

ıg count.

Associated finding IDs
F01
F02, F08, F09, F13
F01, F02, F03, F04, F10
F01, F03, F08, F10
F01, F03, F08, F10
F16
F01, F11
F01, F07
F03, F06
F02, F03, F08, F10
No distinct threat identified (installed with no active sets; any attack needs an assumed flaw)
F16
No distinct threat identified (SSO disabled, no active flow)
F02, F06, F08, F09, F13, F14, F15
F01, F04
F03, F05, F15
F04
F02, F08, F09, F14
F12
F01, F11
F01
F02, F08, F13
No distinct threat identified
F06, F14, F15
No distinct threat identified (TLS with certificate validation stated; no client-auth gap established)
F03, F05, F15
No distinct threat identified
F04
F02, F04, F10
F08, F10
F08, F10
F16
F01
F12
F01, F11
F01, F02, F08, F13, F16
F03, F05, F06, F14, F15
F02, F08, F09, F14
F01, F02, F04
F01, F08
F01, F08
F02
F08
F08

F16